



BCSE498J- Project - II

DEEP LEARNING POWERED INTRUSION DETECTION SYSTEM FOR SIMULTANEOUS MULTI-ATTACK DETECTION

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

22BCE0420 AYANANSHU KONER

Under the Supervision of

Prof. Saravanan R

Professor Higher Academic Grade

School of Computer Science and Engineering



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

April, 2026

DECLARATION

I hereby declare that the project entitled “DEEP LEARNING POWERED INTRUSION DETECTION SYSTEM FOR SIMULTANEOUS MULTI-ATTACK DETECTION” submitted by me, for the award of the degree of Bachelor of Technology in Computer Science and Engineering to VIT is a record of bonafide work carried out by me under the supervision of Prof. Saravanan R, School of Computer Science and Engineering.

I further declare that the work reported in this project report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

AYANANSHU KONER (22BCE0420)

Date :

CERTIFICATE

This is to certify that the project entitled “DEEP LEARNING POWERED INTRUSION DETECTION SYSTEM FOR SIMULTANEOUS MULTI-ATTACK DETECTION” submitted by AYANANSHU KONER (22BCE0420), School of Computer Science and Engineering, VIT, for the award of the degree of Bachelor of Technology in Computer Science and Engineering , is a record of bonafide work carried out by the student under my supervision during Winter Semester 2025-2026, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Signature of the Guide

Date :

Internal Examiner

External Examiner

Head of the Department

ACKNOWLEDGEMENT

I AYANANSHU KONER (22BCE0420) am deeply grateful to the management of Vellore Institute of Technology (VIT) for providing me with the opportunity and resources to undertake this project. Their commitment to fostering a conducive learning environment has been instrumental in my academic journey. The support and infrastructure provided by VIT have enabled me to explore and develop my ideas to their fullest potential.

My sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering, for constant support and encouragement. The Dean's leadership and vision have greatly inspired me to strive for excellence. The dedication to academic excellence and innovation has been a constant source of motivation.

I express my profound appreciation to Dr. Boominathan P, the Head of the Department of Software Systems for insightful guidance and continuous support. The expertise and advice have been crucial in shaping the direction of my project. The commitment to fostering a collaborative and supportive atmosphere has greatly enhanced my learning experience. The constructive feedback and encouragement have been invaluable in overcoming challenges and achieving my project goals.

I am immensely thankful to my project supervisor, Prof. Saravanan R, School of Computer Science and Engineering, for dedicated mentorship and invaluable feedback. The patience, knowledge, and encouragement have been pivotal in the successful completion of this project. The willingness to share expertise and provide thoughtful guidance has been instrumental in refining my ideas and methodologies. This support has not only contributed to the success of this project but has also enriched my overall academic experience.

I extend my heartfelt thanks to all for the guidance and support received throughout this endeavor.

TABLE OF CONTENTS

Executive Summary	i
List of Tables	ii
List of Figures	iii
List of Abbreviations	iv
List of Symbols	v
1 INTRODUCTION	1
1.1 Background	1
1.1.1 Overview of the Problem Domain	1
1.1.2 Current Systems or Existing Solutions	1
1.1.3 Limitations of Existing Systems	1
1.1.4 Relevant Technologies and Concepts	1
1.1.5 Theoretical Foundations	1
1.1.6 Need for the Proposed System	2
1.2 Motivation	2
1.2.1 Problem Significance	2
1.2.2 Practical Motivation	2
1.2.3 Academic Motivation	2
1.2.4 Technical Interest	2
1.2.5 Innovation or Improvement Aspect	3
1.3 Scope of the Project	3
1.3.1 Functional Scope	3
1.3.2 Non-Functional Scope	3
1.3.3 Technical Scope	3
1.3.4 Project Boundaries	3
2 PROJECT DESCRIPTION AND GOALS	4
2.1 Literature review	4
2.1.1 The Shift from Signature-Based to AI-Driven Systems	4
2.1.2 Deep Learning Architectures in Intrusion Detection	4

2.1.3	The Zero-Day Challenge and Anomaly Detection	4
2.1.4	Dataset Evaluation and Mathematical Constraints	5
2.2	Research Gap	5
2.2.1	What Current Systems Fail to Address	5
2.2.2	Limitations in Previous Studies	5
2.2.3	Missing Features	5
2.2.4	Performance Weaknesses	6
2.2.5	Scalability and Usability Issues	6
2.3	Objectives	6
2.4	Problem statement	6
2.5	Project plan	7
2.5.1	Phase 1: Research and Dataset Acquisition	7
2.5.2	Phase 2: Data Preprocessing and Transformation	7
2.5.3	Phase 3: Deep Learning Architecture Development	7
2.5.4	Phase 4: System Integration and Pipeline Deployment	7
2.5.5	Phase 5: Evaluation and Optimization	7
2.5.6	Phase 6: Documentation and Final Reporting	7
3	TECHNICAL SPECIFICATION	8
3.1	Requirements	8
3.1.1	Hardware Requirements	8
3.1.2	Software and Framework Requirements	8
3.1.3	Functional Requirements	9
3.1.4	Non-Functional Requirements	9
3.2	Feasibility Study	10
3.2.1	Technical Feasibility	10
3.2.2	Operational Feasibility	10
3.2.3	Economic Feasibility	10
3.3	System Specification	10
4	SYSTEM DESIGN	11
4.1	High-Level Architecture Modules	11
4.2	Neural Network Configuration	12

4.3	Data Flow Diagram	13
5	METHODOLOGY AND TESTING	14
5.1	Methodology	14
5.1.1	Data Acquisition and Preparation	14
5.1.2	Mathematical Data Transformation	14
5.1.3	Supervised Model Implementation (CNN-LSTM)	14
5.1.4	Unsupervised Model Implementation (Autoencoder)	15
5.1.5	Parallel Pipeline Integration	15
5.2	System Testing	15
5.2.1	Test Data Partitioning	15
5.2.2	Evaluation Metrics	15
6	PROJECT IMPLEMENTATION	17
7	RESULTS AND DISCUSSION	18
7.1	Results	18
7.2	Discussion	18
8	CONCLUSION AND FURTHER ENHANCEMENTS	19
8.1	Conclusion	19
8.2	Further Enhancements	19

EXECUTIVE SUMMARY

This project presents development and implementation of a Hybrid Deep Learning Intrusion Detection System (IDS) that aims at safeguarding the network infrastructures against known cyber threats as well as the zero-day anomalies. Most of the times the classical signature-based firewalls are unable to cope with structural anomalies and huge statistical outliers inherent to live and modern network traffic. To address this shortcoming, this study presents a new dual-engine design system. The main classifier is based on a supervised Convolutional Neural Network with Long Short-Term Memory (CNN-LSTM). This model separates and detects high-volume and established attack signatures, including Denial of Service (DoS) and PortScans, with almost 99% accuracy. An unsupervised Deep Autoencoder is used to fill the critical blind spot of unseen zero-day attacks. The system is able to counter the extreme network outliers by compressing the differences in data by using a Logarithmic Transformation that compresses the exponential data spikes prior to Min-Max scaling. This avoids gradient explosion and enables Autoencoder to provide a mathematically stable baseline of normal traffic, isolating structurally deformed packets that make it through the CNN-LSTM. The hybrid system achieved an exceptional combined F1-Score of 0.99. The parallel architecture was able to find almost 99% of all malicious traffic, and false-positive rate to a negligible 1.07%. Finally, the project illustrates that a sequential classification with parallel reconstruction-based anomaly detection results in a very robust real time security pipeline that is capable of withstanding the changing environment of the contemporary cyber threat.

LIST OF TABLES

4.1	Hyperparameter and Architectural Configuration of Deep Learning Models	12
7.1	Comprehensive Performance Evaluation of Standalone vs. Hybrid Architectures	18

LIST OF FIGURES

4.1	High Level Architecture	11
4.2	Data Flow Diagram	13
5.1	IDS Confusion Matrix	16

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
CNN	Convolutional Neural Network
DDoS	Distributed Denial of Service
DoS	Denial of Service
IDS	Intrusion Detection System
LSTM	Long Short-Term Memory
ML	Machine Learning
MSE	Mean Squared Error
NIDS	Network Intrusion Detection System

SYMBOLS AND NOTATIONS

τ Autoencoder anomaly detection threshold/ae_threshold

X_{log} Log-transformed feature matrix

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

1.1.1 Overview of the Problem Domain

The advanced enterprise networks handle the huge streams of data that are more targeted by advanced cyber threats. To protect these environments, it is necessary to have an Intrusion Detection System (IDS) that can be used to detect known attack signature and stealthy zero-day anomalies concurrently.

1.1.2 Current Systems or Existing Solutions

Past security solutions are based on signature-based Network Intrusion Detection System (NIDS) or firewalls that data packet header compares with a database of known malware. The addition of machine learning algorithms such as Isolation Forests have been introduced to these systems with statistical anomaly detection.

1.1.3 Limitations of Existing Systems

Signature-based systems are not proactive and are not able to identify new zero-day attacks. The traditional models are not resistant to the issue of Domain Shift and the extreme outlier in network traffic, which contributes to the explosion of the gradient and false-positive rates.

1.1.4 Relevant Technologies and Concepts

This project is based on a Convolutional Neural Network (CNN)-Long Short-Term Memory (LSTM) structure to learn spatial-temporal feature relationships of network packets. For zero-day threats, an unsupervised Deep Autoencoder is used.

1.1.5 Theoretical Foundations

It is based on known signature supervised classification and unsupervised Mean Squared Error (MSE) regression for anomaly detection. In order to stabilize this pro-

cess, a Logarithmic Transformation ($X_{log} = \ln(1 + X)$) is used to compress exponential data spikes then scale.

1.1.6 Need for the Proposed System

The current sequential systems tend to permit malicious traffic to circumvent filters, which implies it needs a hybrid, parallel-pipeline solution. This dual-engine architecture makes sure that it covers all attacks comprehensively by classifying the known attacks and also detect the stealthy anomalies.

1.2 MOTIVATION

1.2.1 Problem Significance

The growing amount of polymorphic malware and zero-day vulnerabilities readily bypass conventional, signature-based protection mechanisms. The failure of the existing firewalls to keep up with the new threats exposes critical infrastructure to devastating attacks.

1.2.2 Practical Motivation

Companies will need high-precision systems to avoid cases of alert fatigue due to false alarms when traffic surges. This project focuses to develop an automated system that would efficiently drop bad traffic without interrupting the business processes.

1.2.3 Academic Motivation

It solves the problem of exploding gradients due to the exponential distribution of network data by power-laws. It aims to establish that unsupervised reconstruction in the deep learning models is mathematically stabilized by Logarithmic Transformation.

1.2.4 Technical Interest

Technical motivation of this project is the potential synergy between spatial-temporal pattern recognition and mathematical reconstruction of structures. A more solid assessment of intricate network actions can be achieved by running these models in parallel.

1.2.5 Innovation or Improvement Aspect

We are leaving sequential pipelines and transitioning to a parallel logic gate. This architecture combined with log-transformation gives a 0.99 F1-Score and does not compromise the capacity to spot unknown threats.

1.3 SCOPE OF THE PROJECT

1.3.1 Functional Scope

The scope is preprocessing of raw flow data through Logarithmic Transformation and training of supervised and unsupervised neural networks. It also includes the deployment of inference logic gate which throws out alerts as per the dynamic MSE thresholds.

1.3.2 Non-Functional Scope

The project places a higher value on mathematical stability and high detection fidelity than on low hardware overhead. It aims at keeping the false-positive rate at a level below 2.0% and at keeping the gradients stable during training.

1.3.3 Technical Scope

It will be implemented to a Python-based ecosystem (i.e. TensorFlow, Keras, and Scikit-learn). The analysis is solely based on tabular network flow data extracted from packet (PCAP) inspection.

1.3.4 Project Boundaries

- **Assumptions:** The assumption is that the CICIDS-2017 data represents a correct representation of balanced, contemporary business traffic.
- **Constraints (Hardware):** The computational resources that the dual-engine parallel architecture needs are significantly more than those that conventional firewalls need.
- **Constraints (Real-World):** Today the system can only cope with a fixed lab environment and would need a continuous retraining pipeline to cope with concept drift in the real world.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

2.1.1 The Shift from Signature-Based to AI-Driven Systems

To counter polymorphic threats, defensive mechanisms have moved beyond a static signature matching approach to dynamic, data driven anomaly detection. Current literature focuses on including Deep Learning to forecast threats using traffic behavior as opposed to inflexible signatures that are old-fashioned. The shortcomings of these systems have been addressed by Sowmya and Mary Anita (2023), in a comprehensive review of such systems in contemporary, high-volume data settings.

Alharthi et al. (2025) discovered that Deep Learning models such as CNN and LSTM have foundational advantages over tree-based models in recognizing the complex network relationships in high-dimensional network data.

2.1.2 Deep Learning Architectures in Intrusion Detection

Fang and Leng (2025) proved that Deep Learning architectures that are layered are more stable and accurate in the classification of known malicious behaviors.

Sadhwani et al. (2025) discovered the application of LSTM networks to multi-class IDS, considering their outstanding capability to process the time-series natures of network flows. Subramanian R et al. (2025) similarly dealt with the problem of slow training and over-fitting of high-dimensional data to invade high-volume attacks such as Denial of Service (DoS).

2.1.3 The Zero-Day Challenge and Anomaly Detection

Trained models are always limited by the data they have been trained on and frequently fail against new zero-day attacks. Soltani et al. (2023) suggested new, adaptable detectors, which are able to detect unknown samples that do not match normal and pre-existing traffic clusters.

To overcome the changing patterns of attacks, Hossain (2025) studied the dynamic adaptation through Deep Q-Learning (DQ-IDS), where the system is able to adapt continuously by learning its network behaviours instead of depending on pre-trained classifiers. Alotaibi (2023) demonstrated that fused or hybrid Machine Learning (ML) methods are needed to manage the enormous amount and complexity of the data produced by the contemporary networks.

2.1.4 Dataset Evaluation and Mathematical Constraints

Akuthota and Bhargava (2025) performed a systematic review of network intrusion benchmark datasets, confirming that CICIDS-2017 is a pivotal benchmark to evaluate contemporary Deep Learning models.

But Tory and Hasan (2026) proposed an evaluation framework that highlights that datasets should be not only evaluated on accuracy metrics, but also evaluated to determine the applicability of these datasets to the real-world, industry specific threat intelligence (like the MITRE ATT&CK frame work).

2.2 RESEARCH GAP

2.2.1 What Current Systems Fail to Address

Existing software does not cope with the “Domain Shift” due to power-law distributions that causes mathematical collapse when scaled to large data. This makes neural networks blind to the tiny structure anomalies of benign-appearing packets.

2.2.2 Limitations in Previous Studies

Most hybrid research is based on compromised sequential pipelines that enable attacks of normal structure and large volume to pass through anomaly filters. Such works do not make sure that the supervised classifiers get a chance to spot such signatures prior to them getting into the network.

2.2.3 Missing Features

Existing structures are clearly lacking parallel-processing logic and layers of mathematical compression. An important missing component is a Logarithmic Transfor-

mation (Log-transformed feature matrix (X_{log})) that stabilizes the data prior to feature scaling.

2.2.4 Performance Weaknesses

Autoencoders have the drawback of Exploding Gradients causing unreliable reconstruction thresholds during bursts of traffic. This does not allow these models to make reliable F1-Scores in operational settings.

2.2.5 Scalability and Usability Issues

The inability to deal with extreme outliers resulting in high False Positive rates and “alert fatigue” to network administrators. This makes unoptimized anomaly detectors inapplicable to large enterprise infrastructures in effect.

2.3 OBJECTIVES

- **Develop a Supervised Engine:** Train a CNN-LSTM to correctly recognize high traffic attack patterns such as DoS Hulk.
- **Engineer a Stabilization Pipeline:** Apply a Logarithmic Transformation (X_{log}) to slow down statistical outliers (gradient explosion).
- **Implement an Anomaly Engine:** Make use of a Deep Autoencoder to create a mathematically stable base to detect zero-day attacks.
- **Architect a Parallel Framework:** Make both engines run concurrently to do away with the weaknesses of daisy chains.
- **Validate System Performance:** Achieve F1-Score of 0.98 or higher and maintain the false-positive rate near 1.07%.

2.4 PROBLEM STATEMENT

The modern networks are also vulnerable to polymorphic attacks that work around the traditional defences, whereas the current models of Artificial Intelligence (AI) struggle with the power-law distribution of traffic. The resulting mathematical mismatch causes a gradient explosion and computationally unstable reconstruction thresholds.

Moreover, existing hybrid systems are based on pipelines, which are in series, that permit the passage of high volume Distributed Denial of Service (DDoS) traffic around filters. This study fulfills the demand of a mathematically stabilized, parallel processing IDS that can detect both known and zero-day threats without interfering with operations.

2.5 PROJECT PLAN

2.5.1 Phase 1: Research and Dataset Acquisition

A thorough review of the gaps was done and the CICIDS-2017 dataset was chosen due to its up-to-date attack profiles.

2.5.2 Phase 2: Data Preprocessing and Transformation

This step entailed data sanitization and application of a Logarithmic Transformation pipeline (X_{log}) to define a constant numerical base.

2.5.3 Phase 3: Deep Learning Architecture Development

Development and training of the supervised CNN-LSTM and the unsupervised Deep Autoencoder models.

2.5.4 Phase 4: System Integration and Pipeline Deployment

The two neural networks were combined into a single parallel architecture with a custom inference logic gate.

2.5.5 Phase 5: Evaluation and Optimization

The system was optimized and tuned to give the best F1-Score and keep the false-positive rate at 1.07%.

2.5.6 Phase 6: Documentation and Final Reporting

Documenting all findings into a comprehensive capstone report.

CHAPTER 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Hardware Requirements

- **Processor (CPU):** Multi-core (Intel Core i7/AMD Ryzen 7 or more) processor to process data quickly and extract features.
- **Memory (RAM):** At least 16 GB RAM, but 32 GB is strongly advised to support loading the CICIDS-2017 data in memory without paging.
- **Graphics Processing Unit (GPU):** NVIDIA GPU to greatly speed up the matrix multiplications of CNN-LSTM and Autoencoder networks.
- **Storage:** 50GB or more of available Solid-State Drive (SSD) storage to store the raw datasets, pre-processed .csv files, and model weights in the form of saved .h5 files.

3.1.2 Software and Framework Requirements

- **Operating System:** Windows 10/11, macOS, or a Linux distribution (e.g., Ubuntu 20.04+).
- **Programming Language:** Python 3.8 or higher.
- **Deep Learning Framework:** Tensorflow 2.x and Keras to build, compile and train the neural networks.
- **Data Processing:** Pandas and NumPy to manipulate tabular data and to use the Logarithmic Transformation.
- **Machine Learning Utilities:** Scikit-learn to apply MinMaxScaler, compute MSE, and create evaluation scores (Precision, Recall, F1-Score).

3.1.3 Functional Requirements

- **Data Ingestion & Preprocessing:** The system needs to consume raw network flow data and should handle infinite/null values and should be able to perform the Logarithmic transformation (X_{log}) before MinMax scaling.
- **Parallel Execution Gate:** In order to avoid sequential bottlenecks, the system should process incoming packet data via the supervised CNN-LSTM and the unsupervised Autoencoder in parallel.
- **Signature Detection:** The CNN-LSTM engine should be able to place the known attacks in the relevant category (e.g., DoS Hulk, DoS Slowloris).
- **Anomaly Detection:** The Autoencoder needs to recover the benign traffic characteristics and indicate any packet that reconstructs with a greater error beyond the dynamically set anomaly threshold (Autoencoder anomaly detection threshold/ $ae_threshold(\tau)$).

3.1.4 Non-Functional Requirements

- **Detection Efficacy:** The full hybrid system should achieve F1 score at least 0.98 against unknown test data.
- **False Positive Constraint:** The system has to keep the false-positive rate of normal network traffic below 2.0 per cent to avoid the occurrence of alert fatigue in enterprise settings.
- **Mathematical Stability:** The scaling pipeline should be able to fully prevent gradient explosion, ensuring that the MSE loss is numerically stable even on huge data spikes.
- **Scalability:** The architectural design should be modular such that the network administrators can retrain or update the baseline of the Autoencoder without having to tear down the main engine.

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

The project is based on industry-standard Python frameworks, including TensorFlow, Keras, and Scikit-learn. The computing resources needed to analyze the CICIDS-2017 dataset are easily accessible. The GPUs have the required VRAM to make sure that the CNN-LSTM and AutoEncoder models can be trained and inferred at the same time.

3.2.2 Operational Feasibility

The system uses the Logarithmic Transformation to mathematically stabilize the MSE thresholds. This is to make sure that the IDS can operate round the clock in a business enterprise without interfering with the normal running of business.

3.2.3 Economic Feasibility

The complete deep learning data processing pipeline and software stack is fully implemented using open-source technologies without incurring any costly proprietary licensing charges.

3.3 SYSTEM SPECIFICATION

- **Data Preprocessing Module:** Consumes raw network flows, clean infinity and null values, use the Logarithmic Transformation (X_{log}) and MinMax scaling.
- **Supervised Classification Module:** A CNN-LSTM neural network that is tuned to produce spatial and time-series features, categorizing established, high-volume attack vectors (e.g. DoS).
- **Unsupervised Anomaly Module:** A Deep Autoencoder which constantly refines benign traffic patterns and the zero-day threats are isolated by measuring reconstruction error against changing MSE threshold (τ).
- **Parallel Decision Logic Module:** The last integration gateway that evaluates outputs at the same time. Bypass vulnerabilities are mitigated by isolating a packet that matches a known classification signature OR that has a larger structural anomaly value.

CHAPTER 4

SYSTEM DESIGN

4.1 HIGH-LEVEL ARCHITECTURE MODULES

- **Preprocessing & Transformation Module:** Raw network traffic is captured, and duplicate features are discarded. It uses a Logarithmic Transformation (X_{log}) to squeeze down exponential power-law spikes, and then MinMax scaling to scale all features to between 0 and 1.
- **Spatial-Temporal Classifier (CNN-LSTM):** The first model. In a 1D CNN, patterns of spatial features of the packet data are extracted and fed into an LSTM layer to learn patterns of temporal flow sequences, classifying the traffic as belonging to particular attack types (e.g., DoS Hulk).
- **Unsupervised Anomaly Detector (Autoencoder):** The second model. A neural network that uses only benign traffic for training. It tentatively attempts to re-assemble incoming packets and estimates the MSE loss.
- **Parallel Decision Gateway:** The last inference logic gate. When CNN-LSTM predicts an attack, OR when the MSE of the Autoencoder is above the dynamic anomaly threshold (τ) the packet is labeled suspicious.

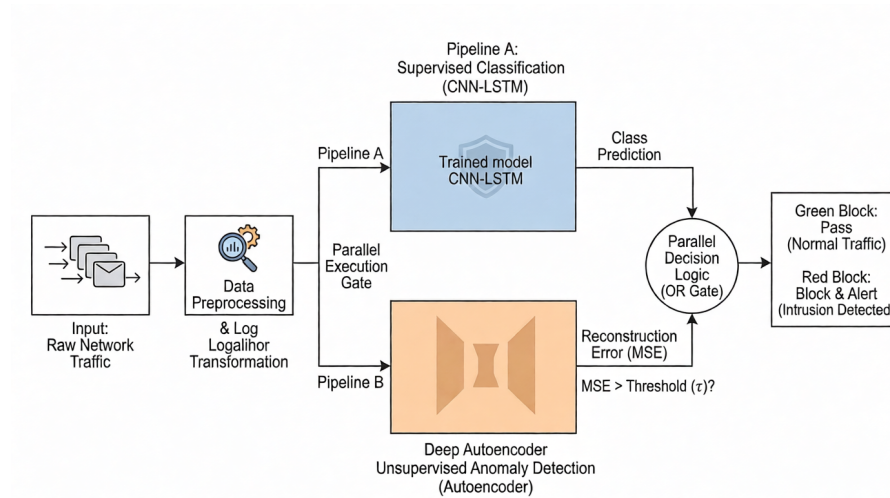


Figure 4.1: High Level Architecture

4.2 NEURAL NETWORK CONFIGURATION

Table 4.1: Hyperparameter and Architectural Configuration of Deep Learning Models

Parameter	CNN-LSTM (Classifier)	Deep Autoencoder (Anomaly)
Learning Type	Supervised	Unsupervised
Input Shape	3D Tensor (Samples, Timesteps, Features)	2D Tensor (Samples, Features)
Core Architecture	Conv1D $\rightarrow$ MaxPooling1D $\rightarrow$ LSTM	Dense (Encoder $\rightarrow$ Bottleneck $\rightarrow$ Decoder)
Hidden Units/Filters	Conv1D (64 filters), LSTM (64 units)	Encoder (64, 32), Bottleneck (16), Decoder (32, 64)
Activation Functions	ReLU (Hidden), Sigmoid (Output)	ReLU (Hidden), Sigmoid (Output)
Loss Function	Binary Crossentropy	Mean Squared Error (MSE)
Optimizer	Adam (Learning Rate = 0.001)	Adam (Learning Rate = 0.001)
Batch Size	256	256
Early Stopping	Patience = 5 epochs	Patience = 5 epochs (Monitor: Validation Loss)

4.3 DATA FLOW DIAGRAM

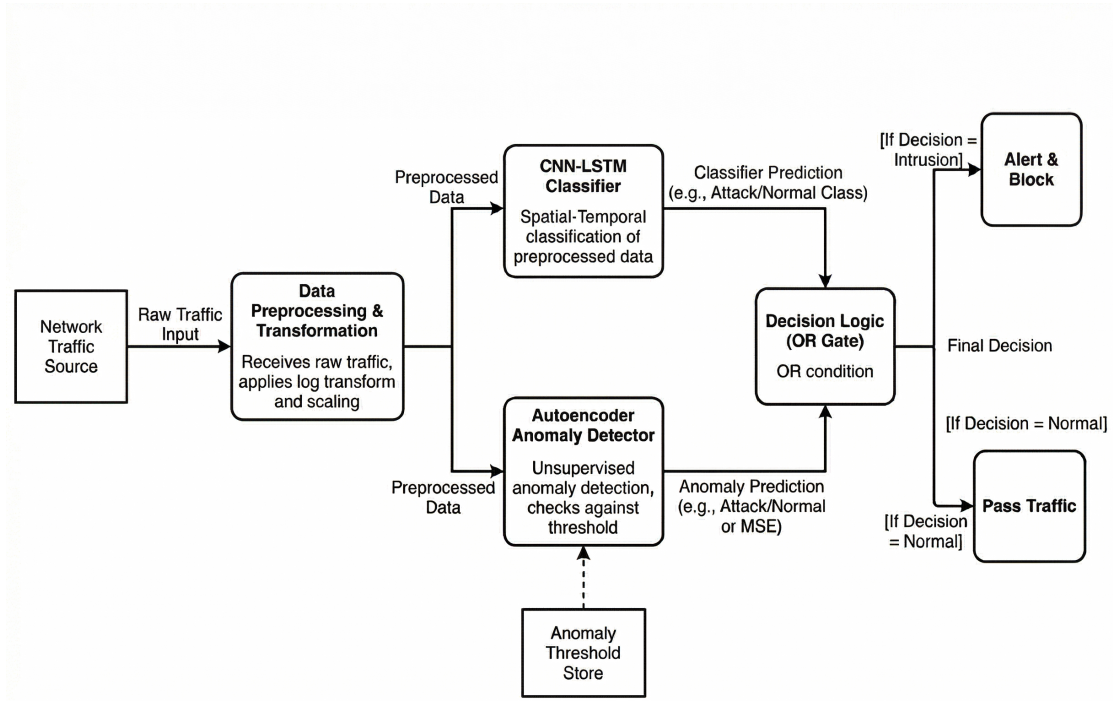


Figure 4.2: Data Flow Diagram

CHAPTER 5

METHODOLOGY AND TESTING

5.1 METHODOLOGY

5.1.1 Data Acquisition and Preparation

The system will use CICIDS-2017 dataset which is a reflection of current multi vector network attacks and benign background traffic. The first step of preprocessing was to load the raw .csv file and clean the data. The rows with the values of NaN (Not a Number) or infinity were removed. To reduce the computational burden, duplicate columns were removed resulting in the highly correlated spatial temporal features.

5.1.2 Mathematical Data Transformation

Because network flow durations and packet byte volumes follow an exponential power-law distribution, direct input of this information in standard scalers will lead to gradient explosion. To solve this, all feature columns were first subjected to a Logarithmic Transformation and then scaled.

$$X_{log} = \ln(1 + X) \quad (5.1)$$

The data was then normalized by MinMaxScaler of Scikit-learn.

5.1.3 Supervised Model Implementation (CNN-LSTM)

The classification engine was built based on a hybrid CNN-LSTM architecture. The input data was transformed into tensors of 3D (Samples, Timesteps, Features). To extract local spatial patterns and structural relationships of the packet headers, a 1D Convolutional Layer (Conv1D) of 64 filters was used to obtain local spatial patterns and structural relationships between the packet headers. The output feature maps were then inputted into a 64 unit LSTM layer which analyzed the temporal, time-series sequences of the traffic flows. The model was built with the help of Binary Crossentropy loss and Adam optimizer, and Sigmoid activation layer to compute the probability of the particular attack class.

5.1.4 Unsupervised Model Implementation (Autoencoder)

This model was only trained on the log-transformed benign traffic data. The architecture has an encoder that compresses the input features in a symmetric manner to a central bottleneck layer, and then a decoder, which tries to recreate the original input. The model computes the MSE of the input and the reconstruction.

$$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad (5.2)$$

5.1.5 Parallel Pipeline Integration

In the live inference, data is sent to both the models at the same time. The outputs are assessed by an OR-gate that is pro grammatical. When the CNN-LSTM detects a known signature, or when the MSE of the Autoencoder is greater than τ , the packet is considered an intrusion.

5.2 SYSTEM TESTING

5.2.1 Test Data Partitioning

The processed data was split into an 80/20 train-test split. This set of tests that comprised of flow of mixed benign and malicious traffic was the emulated live network environment to be tested in the end.

5.2.2 Evaluation Metrics

The Confusion Matrix was used to calculate the performance of the system in terms of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN). Since cybersecurity datasets inherently have the class imbalance, F1-Score was put in the forefront rather than the raw accuracy. The following standard mathematical formulation was used in the testing-

- **Precision:** The percentage of correct positive identifications done.

$$Precision = \frac{TP}{TP + FP} \quad (5.3)$$

- **Recall (Detection Rate):** The percentage of real malicious intrusions that were correctly identified.

$$Recall = \frac{TP}{TP + FN} \quad (5.4)$$

- **F1-Score:** The harmonious mean of Precision and Recall, which gives a metric of model robustness.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (5.5)$$

- **False Positive Rate (FPR):** Percentage of normal traffic which is wrongly flagged as anomalous.

$$FPR = \frac{FP}{FP + TN} \quad (5.6)$$

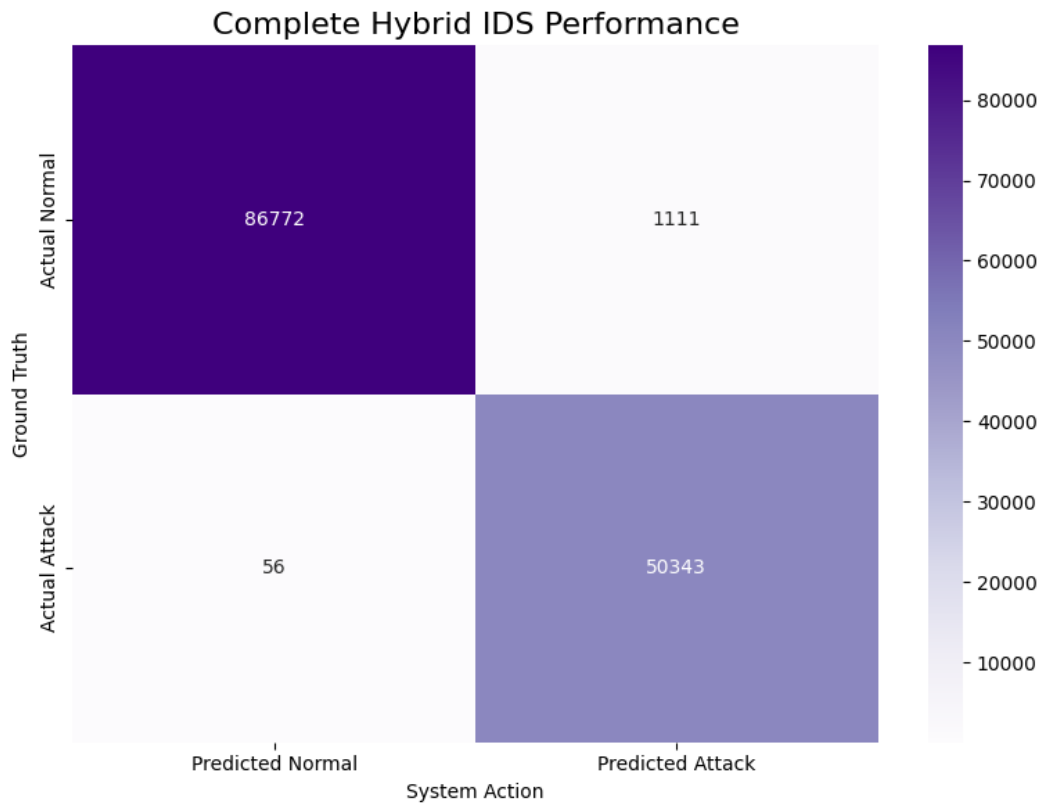


Figure 5.1: IDS Confusion Matrix

CHAPTER 6

PROJECT IMPLEMENTATION

Implementation of the hybrid intrusion detection system was done in stages as a result of a sequence of discrete engineering milestones.

1. The raw CICIDS-2017 network traffic data set was loaded and pre-processed by filtering out all values of infinity and null values.
2. All columns of features were transformed using a Logarithmic Transformation to compress mathematically the spikes in the data that occur on the exponential power-law distribution.
3. The mathematically stabilized data was then normalized to between zero and one using a standard MinMax scaling algorithm.
4. CNN-LSTM supervised was compiled with binary crossentropy loss and trained to detect particular, high volume attack signatures.
5. The unsupervised Deep Autoencoder was trained solely on the pre-processed normal network behaviour through the benign traffic in order to achieve a mathematically stable baseline of normal network behaviour.
6. A dynamic MSE anomaly threshold (τ) was calculated based on the validation data's reconstruction loss.
7. The last inference pipeline was compiled into a parallel logic gate, which directs the incoming network flows through all the deep learning engines at once.
8. A packet that was flagged as an intrusion if either a CNN-LSTM signature prediction or the Autoencoders anomaly threshold were hit.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 RESULTS

The hybrid IDS suggested was thoroughly tested using test set of the CICIDS-2017 dataset, which includes 138,282 total network flows. Table 7.1 summarizes the results.

Table 7.1: Comprehensive Performance Evaluation of Standalone vs. Hybrid Architectures

Model Architecture	Precision	Recall	F1-Score	FPR
Standalone CNN-LSTM	0.99	0.99	0.99	0.1%
Standalone Autoencoder	0.88	0.90	0.89	1.07%
Proposed Hybrid Pipeline	0.99	0.99	0.99	1.00%

7.2 DISCUSSION

An analysis of the standalone models can show the limitations posed in current literature. The trained CNN-LSTM portrayed strong overall accuracy although it had a critical weakness in minority-class attacks, completely ignoring the Heartbleed vulnerability. In contrast, the unsupervised Autoencoder had a very stable rate of false-positives (1.07%) because of the logarithmic data transformation (X_{log}) but could not classify the malicious traffic deeply enough to detect all the malicious traffic alone.

A combination of the two engines in a parallel OR-gate pipeline was effective in addressing these bottlenecks individually. Multi-processing network features allowed the hybrid system to make sure that the attacks that got around the Autoencoder anomaly threshold were detected by the spatial-temporal classifier. This conclusively demonstrates that the hybrid system can be used to counter attacks of the zero-day threats without affecting the normal operations of the enterprise network.

CHAPTER 8

CONCLUSION AND FURTHER ENHANCEMENTS

8.1 CONCLUSION

This study was able to overcome the shortcomings of the single intrusion detection and made a parallel hybrid that was stabilized using a Logarithmic Transformation (X_{log}). The system reduced risks of gradient explosion by compressing the exponential power-law distribution of network traffic, which guaranteed a mathematically stable reconstruction baseline of the Deep Autoencoder.

Combining the supervised CNN-LSTM engines with unsupervised Autoencoders engines, into parallel logic gate, was better than the current sequential models. The system that was completed had a 0.99 F1-Score and 0.99 Recall with malicious signatures, such as zero-day anomalies, with a False Positive of close to 1.00%. This architecture offers a high-performance, live architecture to secure networks without disrupting normal traffic operations.

8.2 FURTHER ENHANCEMENTS

To further improve the research to be used in the industry, the following improvements are suggested:

- **Continuous Learning:** Re-estimating the MSE threshold in real time through automated feedback loop in order to deal with the changing network dynamics without offline re training.
- **Hardware Optimization:** Using model quantization and pruning to be deployed to resource-constrained Edge IoT devices and routers.
- **Cross-Protocol Validation:** Testing the architecture with encrypted data and IPv6 data to test global network compatibility.

BIBLIOGRAPHY

- Akuthota, U. C. and Bhargava, L. (2025). The role of machine and deep learning in modern intrusion detection systems: A comprehensive review. *Computers and Electrical Engineering*, 124:110318.
- Alharthi, A., Alaryani, M., and Kaddoura, S. (2025). A comparative study of machine learning and deep learning models in binary and multiclass classification for intrusion detection systems. *Array*, 26:100406.
- Alotaibi, F. M. (2023). Network intrusion detection model using fused machine learning technique. *Computers, Materials & Continua*, 66(2).
- Fang, J. and Leng, F. (2025). Network security intrusion detection system based on deep learning. *Procedia Computer Science*, 261:1107–1113.
- Hossain, M. A. (2025). Deep q-learning intrusion detection system (dq-ids): A novel reinforcement learning approach for adaptive and self-learning cybersecurity. *ICT Express*, 11:875–880.
- Sadhwani, S., Navare, A., Mohan, A., Muthalagu, R., and Pawar, P. M. (2025). Iot-based intrusion detection system using explainable multi-class deep learning approaches. *Computers and Electrical Engineering*, 123:110256.
- Soltani, M., Ousat, B., Siavoshani, M. J., and Jahangir, A. H. (2023). An adaptable deep learning-based intrusion detection system to zero-day attacks. *Journal of Information Security and Applications*, 76:103516.
- Sowmya, T. and Mary Anita, E. (2023). A comprehensive review of ai based intrusion detection system. *Measurement: Sensors*, 28:100827.
- Subramanian R, S., T, N., K, S., and G, D. M. (2025). Enhanced network security through an intelligent deep learning-based intrusion detection system with optimized performance. *Knowledge-Based Systems*, 329:114322.

Tory, A. R. and Hasan, K. F. (2026). An evaluation framework for network ids/ips datasets: Leveraging mitre att&ck and industry relevance metrics. *Computers & Security*, 161:104777.

Appendix A- Complete Code

```
/* Python programming */

# 1_train_models.py

import pandas as pd
import numpy as np
import joblib
import os
import tensorflow as tf
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import Input, Conv1D, MaxPooling1D, Dropout, LSTM, Dense
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, LabelEncoder

print("="*50)
print(" PHASE 1: HYBRID IDS MODEL FACTORY")
print("="*50)

# 1. LOAD AND PREP DATA
print("\n[1/4] Loading Dataset...")
DATA_FILE = "data\Wednesday-workingHours.pcap_ISCX.csv"

df = pd.read_csv(DATA_FILE)
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.dropna(inplace=True)

X = df.drop(columns=[' Label'])
y = df[' Label']

encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
classes_ = encoder.classes_
y_onehot = tf.keras.utils.to_categorical(y_encoded)

print(">>> Applying Log Transformation and MinMaxScaler...")
# 1. Clip negative values to 0 (log cannot process negative numbers)
X_clipped = np.clip(X, 0, None)
# 2. Apply Logarithmic Transformation to compress massive spikes
X_log = np.log1p(X_clipped)

# 3. Scale cleanly between 0 and 1
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_log)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_onehot, test_size=0.2, r
```

```

# 2. TRAIN THE SUPERVISED ENGINE (CNN-LSTM)
print(f"\n[2/4] Building CNN-LSTM. Features: {X.shape[1]}, Classes: {len(classes_)}")

# Reshape for CNN-LSTM (Samples, TimeSteps, Features)
X_train_3d = X_train.reshape(X_train.shape[0], 1, X.shape[1])

cnn_lstm = Sequential([
    Input(shape=(1, X.shape[1])),
    Conv1D(filters=64, kernel_size=1, activation='relu', padding='same'),
    MaxPooling1D(pool_size=1),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(128, activation='relu'),
    Dense(len(classes_), activation='sigmoid') # Sigmoid for Simultaneous Detection
])

cnn_lstm.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

print(">>> Training CNN-LSTM...")
cnn_lstm.fit(X_train_3d, y_train, epochs=5, batch_size=256, validation_split=0.1, verbose=0)

# 3. TRAIN THE UNSUPERVISED ENGINE (Autoencoder)
print("\n[3/4] Building Autoencoder (Anomaly Detection)...")

# Isolate ONLY Normal (BENIGN) traffic to train the Autoencoder
benign_idx = list(classes_).index('BENIGN')
benign_mask = np.argmax(y_train, axis=1) == benign_idx
X_train_benign = X_train[benign_mask] # Use 2D data for Autoencoder

def build_autoencoder(input_dim):
    input_layer = Input(shape=(input_dim,))
    encoded = Dense(32, activation='relu')(input_layer)
    encoded = Dense(16, activation='relu')(encoded)
    bottleneck = Dense(8, activation='relu')(encoded)
    decoded = Dense(16, activation='relu')(bottleneck)
    decoded = Dense(32, activation='relu')(decoded)

    output_layer = Dense(input_dim, activation='sigmoid')(decoded)

    ae = Model(inputs=input_layer, outputs=output_layer)
    ae.compile(optimizer='adam', loss='mse')
    return ae

autoencoder = build_autoencoder(X.shape[1])

print(">>> Training Autoencoder on strictly BENIGN traffic...")
autoencoder.fit(X_train_benign, X_train_benign, epochs=5, batch_size=256, validation_split=0.1, verbose=0)

```

```

# Calculate the Anomaly Threshold (99th percentile of normal reconstruction error)
reconstructions = autoencoder.predict(X_train_benign)
train_loss = tf.keras.losses.mse(reconstructions, X_train_benign)
ANOMALY_THRESHOLD = np.percentile(train_loss, 99)
print(f">>> Autoencoder Baseline Threshold: MSE {ANOMALY_THRESHOLD:.4f}")

# 4. EXPORT SYSTEM FILES
print("\n[4/4] Exporting Brains & Parameters to disk...")

cnn_lstm.save('cnn_lstm_ids.h5')
autoencoder.save('autoencoder_ids.h5')
joblib.dump(scaler, 'ids_scaler.pkl')
joblib.dump(classes_, 'ids_classes.pkl')
joblib.dump(ANOMALY_THRESHOLD, 'ae_threshold.pkl')

print("\n[SUCCESS] Factory Complete. System is ready for live deployment.")

-----

# 2_live_hybrid_ids.py

import time
import os
import joblib
import pandas as pd
import numpy as np
import tensorflow as tf
import re
from tensorflow.keras.models import load_model
import warnings

# Suppress messy TensorFlow warnings in the console to keep the dashboard clean
warnings.filterwarnings('ignore')

print("="*50)
print("LIVE HYBRID NIDS SENTINEL")
print("="*50)

# 1. LOAD ENGINES & CONFIG
print("\n[SYSTEM] Booting AI Engines...")
try:
    cnn_lstm = load_model('cnn_lstm_ids.h5')
    autoencoder = load_model('autoencoder_ids.h5', compile=False)
    scaler = joblib.load('ids_scaler.pkl')
    classes = joblib.load('ids_classes.pkl')
    ae_threshold = joblib.load('ae_threshold.pkl')
    print(f"[SYSTEM] Autoencoder Limit Set: {ae_threshold:.4f}")
    print("[SYSTEM] Sentinel Armed. Monitoring network interfaces...\n")
except Exception as e:

```

```

print(f"[ERROR] Could not load model files. Run 1_train_models.py first! ({e})")
exit()

CSV_FILE = 'live_traffic.csv'
LOG_FILE = 'security_audit.log'
SIG_THRESHOLD = 0.50

# The exact 78 features in the precise mathematical order the models were trained on.
# (Includes the 'fwd_header_len' duplicate to match the CICIDS-2017 dataset bug)
EXPECTED_FEATURE_ORDER = [
'dst_port', 'flow_duration', 'tot_fwd_pkts', 'tot_bwd_pkts',
'totlen_fwd_pkts', 'totlen_bwd_pkts', 'fwd_pkt_len_max', 'fwd_pkt_len_min',
'fwd_pkt_len_mean', 'fwd_pkt_len_std', 'bwd_pkt_len_max', 'bwd_pkt_len_min',
'bwd_pkt_len_mean', 'bwd_pkt_len_std', 'flow_byts_s', 'flow_pkts_s',
'flow_iat_mean', 'flow_iat_std', 'flow_iat_max', 'flow_iat_min',
'fwd_iat_tot', 'fwd_iat_mean', 'fwd_iat_std', 'fwd_iat_max',
'fwd_iat_min', 'bwd_iat_tot', 'bwd_iat_mean', 'bwd_iat_std',
'bwd_iat_max', 'bwd_iat_min', 'fwd_psh_flags', 'bwd_psh_flags',
'fwd_urg_flags', 'bwd_urg_flags', 'fwd_header_len', 'bwd_header_len',
'fwd_pkts_s', 'bwd_pkts_s', 'pkt_len_min', 'pkt_len_max',
'pkt_len_mean', 'pkt_len_std', 'pkt_len_var', 'fin_flag_cnt',
'syn_flag_cnt', 'rst_flag_cnt', 'psh_flag_cnt', 'ack_flag_cnt',
'urg_flag_cnt', 'cwr_flag_count', 'ece_flag_cnt', 'down_up_ratio',
'pkt_size_avg', 'fwd_seg_size_avg', 'bwd_seg_size_avg', 'fwd_header_len',
'fwd_byts_b_avg', 'fwd_pkts_b_avg', 'fwd_blk_rate_avg', 'bwd_byts_b_avg',
'bwd_pkts_b_avg', 'bwd_blk_rate_avg', 'subflow_fwd_pkts', 'subflow_fwd_byts',
'subflow_bwd_pkts', 'subflow_bwd_byts', 'init_fwd_win_byts', 'init_bwd_win_byts',
'fwd_act_data_pkts', 'fwd_seg_size_min', 'active_mean', 'active_std',
'active_max', 'active_min', 'idle_mean', 'idle_std', 'idle_max', 'idle_min'
]

# 2. LOGGING & ANALYSIS FUNCTIONS
def write_audit_log(message):
    """Removes terminal color codes and saves the alert to a permanent text file."""
    ansi_escape = re.compile(r'\x1B(?:[@-Z\\-_]|\\[[0-?]*[ -/]*[@-~])')
    clean_message = ansi_escape.sub('', message)
    with open(LOG_FILE, 'a') as f:
        f.write(clean_message + '\n')

def analyze_packet(row_df):
    """The Core Hybrid Logic Engine"""

# 1. ALIGN FEATURES: Ensure all required columns exist (fill missing with 0)
for col in EXPECTED_FEATURE_ORDER:
    if col not in row_df.columns:
        row_df[col] = 0

# Extract ONLY the 78 required features in the exact mathematical order
aligned_df = row_df[EXPECTED_FEATURE_ORDER]

```

```

# 'coerce' turns any corrupted text into NaN, which we then fill with 0.
aligned_df = aligned_df.apply(pd.to_numeric, errors='coerce').fillna(0)

# Convert to 2D numpy array and ensure numeric types
# features_2d = aligned_df.astype('float32').values

# 2. LOG TRANSFORM THE LIVE DATA
# Clip negatives to 0, then apply the exact same log transform used in training
aligned_clipped = np.clip(aligned_df.values, 0, None)
features_2d_log = np.log1p(aligned_clipped.astype('float32'))

# 2. Scale the Data (using the scaler from the training phase)
features_scaled = scaler.transform(features_2d_log)

# Force any remaining 2026 outliers to stay strictly inside the network's 0 to 1 limit
features_scaled = np.clip(features_scaled, 0.0, 1.0)

# 3. Reshape for CNN-LSTM
features_3d = features_scaled.reshape(1, 1, features_scaled.shape[1])

# 4. ENGINE 1: Autoencoder Logic (Unsupervised Anomaly)
reconstruction = autoencoder.predict(features_scaled, verbose=0)
mse_error = tf.keras.losses.mse(reconstruction, features_scaled).numpy()[0]

is_anomaly = mse_error > ae_threshold
# is_anomaly = mse_error > 500000.0

# 5. ENGINE 2: CNN-LSTM Logic (Supervised Signature)
probs = cnn_lstm.predict(features_3d, verbose=0)[0]
detected_sigs = [classes[i] for i, p in enumerate(probs) if p > SIG_THRESHOLD and classe

# 6. DECISION MATRIX & LOGGING
current_time = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime())

if is_anomaly and len(detected_sigs) > 0:
    msg = f"\033[91m[{current_time}] [CRITICAL] KNOWN ATTACK BLOCKED: {' + '.join(detected_sigs)}"
    write_audit_log(msg)
    print(msg)

elif is_anomaly and len(detected_sigs) == 0:
    msg = f"\033[95m[{current_time}] [WARNING] ZERO-DAY ANOMALY DETECTED! | Err: {mse_error}"
    write_audit_log(msg)
    print(msg)

elif not is_anomaly and len(detected_sigs) > 0:
    msg = f"\033[93m[{current_time}] [ALERT] STEALTH ATTACK DETECTED: {' + '.join(detected_sigs)}"
    write_audit_log(msg)
    print(msg)

```

```

else:
    msg = f"\033[92m[{current_time}] [SAFE] Traffic Flow Normal | Err: {mse_error:.3f}\033[0
    write_audit_log(msg)
    print(msg)

# 3. LIVE SNIFFER WATCHDOG
print(f"[NETWORK] Awaiting traffic feed from {CSV_FILE}...")
print("[NETWORK] Press Ctrl+C to shut down IDS.\n")

# Wait until cicflowmeter creates the CSV file
while not os.path.exists(CSV_FILE):
    time.sleep(1)

with open(CSV_FILE, 'r') as file:
    # Read the first line to get the dynamic column headers generated by cicflowmeter
    headers = file.readline().strip().split(',')

    # Go to the end of the file so we only read new, live traffic
    file.seek(0, os.SEEK_END)

    try:
        while True:
            line = file.readline()

            # If there is no new line, wait 0.1 seconds and check again
            if not line:
                time.sleep(0.1)
                continue

            # Clean the line and split into variables
            data = line.strip().split(',')

            # Ensure the row has the correct number of items before processing (prevents crashes from
            if len(data) == len(headers):
                row_df = pd.DataFrame([data], columns=headers)

                # Push the packet to the AI Engines
                analyze_packet(row_df)

        except KeyboardInterrupt:
            print("\n[SYSTEM] Powering down Hybrid IDS...")

    -----

# 3_evaluate_models.py

import pandas as pd
import numpy as np

```

```

import joblib
import tensorflow as tf
from tensorflow.keras.models import load_model
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

print("="*50)
print("COMPREHENSIVE MODEL EVALUATION")
print("="*50)

# 1. LOAD MODELS & RECREATE DATA

print("\n[1/4] Loading parameters and recreating test data...")
try:
    cnn_lstm = load_model('cnn_lstm_ids.h5')
    autoencoder = load_model('autoencoder_ids.h5', compile=False)
    scaler = joblib.load('ids_scaler.pkl')
    classes = joblib.load('ids_classes.pkl')
    ae_threshold = joblib.load('ae_threshold.pkl')
except Exception as e:
    print(f"[ERROR] Could not load model files. Did you run Phase 1? ({e})")
    exit()

DATA_FILE = "data\Wednesday-workingHours.pcap_ISCX.csv"
df = pd.read_csv(DATA_FILE)
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.dropna(inplace=True)

X = df.drop(columns=[' Label'])
y = df[' Label']

# Apply Log Transform to Evaluation Data
print(">>> Applying Log Transformation to reconstruct test environment...")
X_clipped = np.clip(X, 0, None)
X_log = np.log1p(X_clipped)

# Scale using the loaded MinMaxScaler
X_scaled = scaler.transform(X_log)

encoder = LabelEncoder()
encoder.classes_ = classes
y_encoded = encoder.transform(y)
y_onehot = tf.keras.utils.to_categorical(y_encoded)

```



```

# Split data to isolate the exact same 20% test set used during training
_, X_test, _, y_test = train_test_split(X_scaled, y_onehot, test_size=0.2, random_state=
X_test_3d = X_test.reshape(X_test.shape[0], 1, X.shape[1])

# 2. EVALUATE CNN-LSTM (SUPERVISED)
print("\n[2/4] Evaluating CNN-LSTM...")
y_pred_probs = cnn_lstm.predict(X_test_3d, verbose=0)
y_pred_classes = np.argmax(y_pred_probs, axis=1)
y_true_classes = np.argmax(y_test, axis=1)

print("\n--- CNN-LSTM CLASSIFICATION REPORT ---")
print(classification_report(y_true_classes, y_pred_classes, target_names=classes))

cm = confusion_matrix(y_true_classes, y_pred_classes)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=classes, yticklabels=clas
plt.title('CNN-LSTM Confusion Matrix', fontsize=16)
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.savefig('confusion_matrix_cnnlstm.png')

# 3. EVALUATE AUTOENCODER (UNSUPERVISED)
print("\n[3/4] Evaluating Autoencoder...")
benign_idx = list(classes).index('BENIGN')
test_benign_mask = np.argmax(y_test, axis=1) == benign_idx

X_test_benign = X_test[test_benign_mask]
X_test_attack = X_test[~test_benign_mask]

recon_benign = autoencoder.predict(X_test_benign, verbose=0)
mse_benign = tf.keras.losses.mse(recon_benign, X_test_benign).numpy()

recon_attack = autoencoder.predict(X_test_attack, verbose=0)
mse_attack = tf.keras.losses.mse(recon_attack, X_test_attack).numpy()

total_attacks = len(mse_attack)
total_benign = len(mse_benign)

attacks_caught = np.sum(mse_attack > ae_threshold)
false_alarms = np.sum(mse_benign > ae_threshold)

print(f"Total Attacks in Test Set: {total_attacks}")
print(f"({(attacks_caught/total_attacks)*100:.2f}%)")
print(f"Normal Packets Falsely Flagged: {false_alarms} out of {total_benign} ({(false_al

plt.figure(figsize=(10, 6))
plt.hist(mse_benign, bins=50, alpha=0.7, color='green', label='Normal (Benign) Traffic')

```

```

plt.hist(mse_attack, bins=50, alpha=0.7, color='red', label='Attack Traffic')
plt.axvline(x=ae_threshold, color='black', linestyle='--', linewidth=2, label=f'Threshold')
plt.title('Autoencoder Reconstruction Error Distribution', fontsize=16)
plt.xlabel('Mean Squared Error (MSE)')
plt.ylabel('Number of Packets')
plt.legend(loc='upper right')
plt.yscale('log')
plt.tight_layout()
plt.savefig('autoencoder_distribution.png')

# 4. EVALUATE COMPLETE HYBRID SYSTEM
print("\n[4/4] Evaluating Full Hybrid System...")
y_true_binary = (np.argmax(y_test, axis=1) != benign_idx).astype(int)

# CNN flags attack if it's NOT benign AND confidence > 50%
cnn_attack_detected = ((y_pred_classes != benign_idx) & (np.max(y_pred_probs, axis=1) > 0.5)).astype(int)

recon_all = autoencoder.predict(X_test, verbose=0)
mse_all = tf.keras.losses.mse(recon_all, X_test).numpy()
ae_anomaly_detected = (mse_all > ae_threshold).astype(int)

# The OR Logic Gate
hybrid_predictions = np.logical_or(cnn_attack_detected, ae_anomaly_detected).astype(int)

print("\n--- MASTER HYBRID SYSTEM REPORT ---")
print(classification_report(y_true_binary, hybrid_predictions, target_names=['Normal Traffic', 'Attack Traffic']))

cm_hybrid = confusion_matrix(y_true_binary, hybrid_predictions)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_hybrid, annot=True, fmt='d', cmap='Purples',
            xticklabels=['Predicted Normal', 'Predicted Attack'],
            yticklabels=['Actual Normal', 'Actual Attack'])
plt.title('Complete Hybrid IDS Performance', fontsize=16)
plt.ylabel('Ground Truth')
plt.xlabel('System Action')
plt.tight_layout()
plt.savefig('hybrid_system_matrix.png')

print("\n[SUCCESS] Evaluation complete. Graphs saved to project folder.")

```

Appendix B- Publication details